

# SkillWork Platform: Major Technical Terms & Architectural Concepts

Comprehensive reference guide covering the automated work-allocation paradigm, concurrency control, backend engine, and full-stack architecture.

## 1. Work-Allocation Engine & Concurrency Control

Mechanisms that eliminate manual job searches and prevent race conditions when multiple workers compete for allocations.

### AUTOMATED WORK PUSH ALLOCATION

#### Push-Based Work Distribution vs Pull-Based Gig Marketplaces

Unlike traditional freelance marketplaces (e.g., Upwork, Fiverr) where workers search, bid, and wait, SkillWork operates on a reverse 'push' paradigm. The platform dynamically matches open tasks to qualified, available workers and pushes real-time opportunities directly to their dashboards.

- **Key Mechanism:** Eliminates unpaid bidding and proposal writing time for skilled workers.
- **Key Mechanism:** Replaces the traditional search bar with an automated matching and notification pipeline.

### ATOMIC DATABASE ROW LOCKING & RACE CONDITION PREVENTION

#### SELECT FOR UPDATE / Atomic Conditional UPDATE (available\_slots > 0)

A critical concurrency mechanism that prevents two workers from successfully claiming the same single task slot simultaneously when clicking 'Grab Work' at the exact same millisecond.

- **Key Mechanism:** Executes: UPDATE tasks SET available\_slots = available\_slots - 1 WHERE id = :id AND available\_slots > 0 AND status = 'AVAILABLE'.
- **Key Mechanism:** Database guarantees only 1 transaction gets rowcount == 1 (200 OK); concurrent callers receive rowcount == 0 and an immediate HTTP 409 Conflict.

### CANDIDATE PRIORITY SCORING ALGORITHM

#### Weighted Multi-Factor Deterministic Scoring Engine

A mathematical scoring model used by the Allocation Service to rank eligible workers before dispatching notifications, ensuring top-performing and reliable workers receive priority access.

- **Key Mechanism:** Formula:  $\text{Score} = (\text{Proficiency} \times 35\%) + (\text{Completion Rate} \times 25\%) + (\text{On-Time Rate} \times 15\%) + (\text{Rating} \times 15\%) + \text{Fairness Factor}$ .
- **Key Mechanism:** Includes an anti-monopoly cooldown so a single high performer cannot monopolize all incoming work.

## 8-POINT ELIGIBILITY FILTER ENGINE

### Multi-Gate Pre-Qualification Verification

A strict 8-step verification pipeline in the backend that any worker must satisfy before a task is visible or assignable to them.

- **Key Mechanism:** Gates: 1) Active Account, 2) Task Availability, 3) Open Allocation Window, 4) Worker Available (Online), 5) Exact Skill Match, 6) Verified Proficiency Threshold, 7) Daily Task Limit (< 2/day), 8) No Active Conflicting Task.

## TIERED BATCH NOTIFICATION & ESCALATION WINDOW

### Staged Alert Dispatch with Expiry Timers

Instead of blasting all 10,000 workers at once, work is dispatched in timed tiers. Tier 1 (top candidates) receives an exclusive 5-10 minute window. If unclaimed, background workers escalate the task to Tier 2 (wider pool).

- **Key Mechanism:** Protects worker attention and prevents alert fatigue.
- **Key Mechanism:** Ensures high-urgency tasks are filled quickly while rewarding top reliability.

## 2. Backend Engineering & Data Architecture

High-performance API layer, database design, and asynchronous worker architecture.

## FASTAPI & ASGI (ASYNCHRONOUS SERVER GATEWAY INTERFACE)

### Modern Python Asynchronous Web Framework

The core backend engine running Python 3.11+ using Uvicorn. ASGI enables non-blocking asynchronous request handling with native Python type annotations, achieving performance comparable to NodeJS and Go.

- **Key Mechanism:** Auto-generates interactive Swagger (OpenAPI 3.0) documentation at /docs.
- **Key Mechanism:** Delivers sub-millisecond route dispatching for high-frequency work grabbing.

## PYDANTIC V2 VALIDATION & SERIALIZATION

### High-Speed Rust-Powered Data Modeling

The data parsing and validation core of the backend. Enforces strict type schemas on all incoming requests, query params, and outgoing JSON payloads with compile-time-like guarantees.

- **Key Mechanism:** Eliminates silent data corruption and runtime type errors.
- **Key Mechanism:** Automatically sanitizes inputs against malicious payloads.

## SQLALCHEMY ORM & SQLITE WAL MODE

### Object-Relational Mapping with Write-Ahead Logging

Maps Python classes directly to database tables while preserving ACID transactional integrity. Configured with SQLite WAL (Write-Ahead Logging) for zero-config local development, allowing concurrent readers and writers.

- **Key Mechanism:** Supports production drop-in migration to PostgreSQL without changing business logic.
- **Key Mechanism:** WAL mode eliminates 'database table locked' errors during concurrent worker operations.

## JWT (JSON WEB TOKEN) & BCrypt SECURITY

### Stateless Cryptographic Authentication

Industry-standard authentication architecture. Passwords are salted and hashed using BCrypt (cost factor 12). Client sessions are validated statelessly via signed HS256 JWT tokens containing worker identity and role claims.

- **Key Mechanism:** Enables horizontal scaling without server-side session caches.
- **Key Mechanism:** Protected endpoints require an 'Authorization: Bearer ' header.

## BACKGROUND DAEMON WORKERS (WORKERS & CRON)

### Autonomous Asynchronous Task Processors

Autonomous background services that run concurrently with the API to perform housekeeping without blocking HTTP request threads.

- **Key Mechanism:** Allocation Worker: Monitors unclaimed tasks and promotes them across notification tiers.
- **Key Mechanism:** Task Expiry Worker: Automatically reclaims abandoned or expired tasks and restores available slots.

## 3. Worker Progression & Task Lifecycle Engine

State machines governing the 6-stage lifecycle, performance metrics, and skill unlocks.

### 6-STAGE TASK LIFECYCLE STATE MACHINE

#### Strict Deterministic Workflow State Transitions

Every assigned task moves through a strictly ordered finite state machine: 1) Assigned → 2) In Progress → 3) Submitted → 4) Under Review → 5) Approved → 6) Paid.

- **Key Mechanism:** Enforces business rules at each transition (e.g., cannot submit without file attachments or review notes).
- **Key Mechanism:** Provides transparent audit trails for both worker and platform admin.

### EXPONENTIAL MOVING AVERAGE (EMA) PERFORMANCE TRACKING

#### Dynamic Quality & Reliability Metrics

Worker statistics (On-Time Delivery Rate, Quality Rating, Task Completion Percentage) are calculated dynamically rather than as simple static averages. Recent performance is weighted more heavily to reward continuous improvement.

- **Key Mechanism:** Powers worker progression: Level 1 (Novice) → Level 2 (Competent) → Level 3 (Skilled) → Level 4 (Expert).
- **Key Mechanism:** Higher tiers unlock higher-paying tasks and expanded daily task limits.

### SKILL COMPETENCY SCORING & MCQ QUIZ ENGINE

#### Interactive In-Platform Skill Verification

Workers verify their abilities by taking timed, domain-specific multiple-choice assessments. The engine scores submissions, calculates a percentile proficiency score, and updates the worker's verified skills profile.

- **Key Mechanism:** Only verified skills with score  $\geq$  min\_proficiency qualify for task allocation.
- **Key Mechanism:** Directly links learning and testing to immediate earning unlocks.

## 4. Financial Ledger & Payout Subsystem

Financial accounting models ensuring trust, transparency, and instant settlements.

## IMMUTABLE DOUBLE-ENTRY FINANCIAL LEDGER

### Audit-Proof Transaction Recording

Worker balances are never stored as a simple modifiable integer. Every monetary change (Task Reward, Platform Fee, Withdrawal, Reversal) is stored as an immutable ledger transaction with a timestamp, reference ID, and balance snapshot.

- **Key Mechanism:** Prevents phantom balance updates and unauthorized tampering.
- **Key Mechanism:** Guarantees current balance = sum of credits - sum of debits.

## INSTANT WITHDRAWAL SIMULATION (UPI / IMPS)

### Zero-Wait Worker Settlement

Simulates immediate payouts to worker UPI IDs (e.g., worker@okhdfcbank) or direct bank accounts with zero platform lock-in, reflecting the gig economy requirement for rapid liquidity.

- **Key Mechanism:** Validates available balance vs minimum withdrawal thresholds.
- **Key Mechanism:** Instantly shifts ledger status from PENDING to PROCESSED.

## 5. Frontend Architecture & User Experience

Modern Next.js 14 architecture, state management, and real-time interface design.

## NEXT.JS 14 APP ROUTER & REACT 18

### Modern Component Hierarchy with Nested Layouts

The client-side architecture uses Next.js 14 App Router with TypeScript. Pages and layouts are nested cleanly, isolating UI updates and enabling fast client-side navigation without full-page reloads.

- **Key Mechanism:** Employs 'use client' directives for interactive components (timers, modals, grab buttons).
- **Key Mechanism:** Pure Vanilla CSS architecture avoiding heavy external CSS framework overhead.

## LIVE AVAILABILITY TOGGLE & REAL-TIME POLLING

### Instant Worker State Synchronization

The global live availability switch (■ Available / ■ Offline) in the top navigation bar immediately updates the backend state, gating the allocation engine in real time.

- **Key Mechanism:** Workers toggling offline immediately stop receiving task countdown notifications.
- **Key Mechanism:** Hooks (useAvailability, useNotifications) poll endpoints to maintain UI freshness.

## ATOMIC CTA & OPTIMISTIC UI UPDATES

### Low-Latency User Feedback Loops

When a worker taps 'Grab Work', the UI instantly enters an optimistic loading state with tactile feedback, locking the button against double-clicks while the backend executes the atomic row update.

- **Key Mechanism:** If successful: smooth transition into the 6-stage Task Workspace.
- **Key Mechanism:** If claimed by someone else: immediate graceful alert banner with 409 Conflict explanation.